

LAM7ARA

Sistema de Gestión de Inventario y Ventas

Aplicación de escritorio para punto de venta

TERCERA ENTREGA

Autor:	Bruno Depetris
--------	----------------

Docente Tutor:	Gonzalo Castillo
----------------	------------------

Ciclo Lectivo:	2026
----------------	------

Resultados y conclusiones

Videos

Primera actualización: [Aplicacion Gestion en WINDOWS FORM C# + BASE DATOS SQL](#)

url: <https://www.youtube.com/watch?v=IKaChQBjEfE>

Segunda actualización (versión actual): [VERSION 2 LAM7ARA](#)

url: <https://www.youtube.com/watch?v=iuwpPtDMuDs>

Logros Alcanzados

Durante el desarrollo de esta aplicación logré cumplir todos los requerimientos funcionales. Uno de los mayores retos fue lograr conseguir una forma de mantener la integridad de la base de datos, entendiendo el nivel de riesgo que conlleva tener la base de datos a simple vista, decidí que el sistema guarde la base de datos en ciertas carpetas dentro del APPDATA.

Aplicar patrones como acceso de datos asíncrono y manejo de eventos me permitió construir un sistema estable y usable. Esta experiencia me ayudó a aplicar lo visto en clases desde la comunicación con el cliente hasta la implementación final.

Dificultades Enfrentadas

A lo largo del proyecto se presentaron distintos desafíos técnicos que requirieron investigación y solución específica:

Dificultad	Descripcion	Solucion aplicada
Transacciones SQLite multi tabla	La operación de concretar venta requiere insertar en Ventas, insertar en Venta_Productos y actualizar Stock de cada producto, garantizando atomicidad completa.	Se implemento SQLiteTransaction en VentaService.Agregar(), con rollback automático ante cualquier error, incluyendo validación previa de stock suficiente.
normalización de la base de datos	El esquema inicial tenía relaciones rotas: Aperturas tenía VentaID como columna (rompiendo normalización), y Venta_Producto no tenía Cantidad ni PrecioUnitario (snapshot del precio).	Se rediseño el esquema: Cajas absorbe apertura y cierre en una sola tabla con Estado (Abierta/Cerrada); Venta_Productos incorpora Cantidad, PrecioUnitario y Subtotal para preservar el precio histórico.

integración de la API de cotizaciones	Las llamadas HTTP son asíncronas (async/await) pero los formularios Windows Forms tienen restricciones en el manejo del contexto de sincronización.	Se utilizaron métodos async void para los handlers de eventos de botón y async Task para los métodos de carga, garantizando que la UI no se bloquee durante la consulta a la API.
gestión de la base de datos en App Data	La BD debe copiarse del directorio del proyecto a AppData al primer inicio y mantenerse actualizada, con soporte de backup. Esto con la finalidad de que el usuario no tenga fácil acceso.	La clase Conectar centraliza toda la lógica: Comprobar() crea la carpeta y copia la BD; CrearBackUp() genera el respaldo; ObtenerConexion() siempre apunta a la ruta de AppData.

satisfacción de los Requerimientos

El sistema LAM7ARA satisface la totalidad de los requerimientos funcionales establecidos en la primera entrega. Todos los módulos planificados fueron implementados y son completamente operativos: gestión de productos con alertas de stock, gestión de clientes, registro de ventas con múltiples productos por operación, caja con apertura y cierre, dashboard con cotización del dólar y generación de comprobantes PDF.

Los requerimientos no funcionales también se cumplieron: las consultas a SQLite usan parámetros para prevenir inyección SQL, la interfaz mantiene el dark theme consistente con RealTajizor en todos los formularios, la arquitectura en capas (Models / Services / Forms) facilita el mantenimiento, y la integridad transaccional se garantiza mediante SQLiteTransaction en las operaciones de venta.

Impacto Potencial del Software

LAM7ARA representa una solución concreta y lista para usar en comercios minoristas de tecnología del mercado argentino. Su principal diferencial es la integración nativa con la cotización del dólar (oficial, blue y tarjeta), lo que permite al operador tomar decisiones de precios en tiempo real.

La aplicación reduce significativamente el tiempo de atención al cliente al centralizar la búsqueda de productos y clientes, el registro de la venta y la emisión del comprobante en un único flujo. El control automático de stock al concretar una venta elimina errores manuales y proporciona visibilidad inmediata del inventario disponible.

El uso de SQLite como motor de base de datos local garantiza que la aplicación funcione sin dependencias de red o servidores externos, haciendo la resiliente y apropiada para el contexto de conectividad variable de comercios pequeños y medianos.

Trabajo Futuro

A continuación, se detallan las mejoras y extensiones planificadas para versiones futuras de LAM7ARA:

Mejora	Descripción
Sistema de autenticación y roles	Implementar login con tabla Usuarios y roles (Administrador, Vendedor) para controlar el acceso a módulos sensibles como configuración de precios y reportes financieros.
Reportes y exportación a Excel	Generar reportes periódicos de ventas, rentabilidad por producto y evolución de stock exportables a Excel (xlsx) mediante la librería ClosedXML.
sincronización entre equipos	Migrar la base de datos de SQLite local a un servidor compartido (SQL Server o PostgreSQL) para permitir operación multi-equipo dentro del mismo local.
Notificaciones por WhatsApp o email	Integrar envío automático de notificaciones al dueño cuando el stock de algún producto llega a cero, o al cierre de caja con el resumen del día.
aplicación web o multiplataforma	Si en el futuro se requiere compatibilidad con otros sistemas operativos o acceso remoto, la arquitectura de servicios actual facilita la migración a una API REST con frontend web (Blazor, React) o una app multiplataforma (MAUI).

Escalabilidad del Sistema

La arquitectura actual en tres capas (Models, Services, Forms) facilita la escalabilidad del sistema. Agregar un nuevo módulo implica crear el modelo correspondiente, implementar su servicio con los métodos CRUD estándar y desarrollar el formulario que lo consume, sin modificar el resto de la aplicación.

La clase Conectar centraliza el acceso a datos, por lo que migrar de SQLite a otro motor (SQL Server, PostgreSQL) requeriría únicamente actualizar el proveedor en esa clase y ajustar la sintaxis de consultas específicas, sin afectar la capa de servicios ni la de presentación.

Posibilidades de evolución

- migración a arquitectura cliente-servidor: la capa de servicios puede exponerse como una API REST (ASP.NET Core Web API) permitiendo que múltiples clientes se conecten al mismo backend.
- aplicación web o multiplataforma: con los servicios ya definidos, el frontend podría Re implementarse en Blazor WebAssembly, React o .NET MAUI para soportar otros sistemas operativos y dispositivos móviles.
- integración con sistemas de facturación AFIP: agregar soporte para emisión de facturas electrónicas a través de las web cervices de AFIP, reemplazando el comprobante actual no fiscal.
- Módulo de reportes avanzados: generación de informes de rentabilidad, rotación de inventario y proyecciones de ventas con graficos exportables a PDF o Excel.